Technische Universität Braunschweig

A Branch-and-Bound Method for Bilevel Parallel Machine Scheduling with Adversarial Job Selection

Master Thesis Presentation

Ole Damke, April 2026
Institut für Mathematische Optimierung
Technische Universität Braunschweig

Outline

- Motivation & Problem
- Branch-and-Bound Algorithm
- The Ceiling Bound
- The Max-LPT Bound
- Early Optimality Detection
- Computational Experiments
- Conclusion

Motivation

Scenario: production facility

- Manager decides which jobs to accept (budget constraint)
- Scheduler assigns jobs to machines — minimises completion time
- Manager needs a worst-case guarantee on the makespan

This is a **bilevel optimisation problem:**

- **Leader** (manager): selects jobs to maximise makespan
- **Follower** (scheduler): minimises makespan

Our problem

Upper level (Leader)
Unbounded knapsack

$\downarrow$ job portfolio

Lower level (Follower)
Parallel machine scheduling ($P||C_{max}$)

Bilevel Optimisation: Definition

Bilevel Optimisation Problem

$$\max_{x} \quad F\big(x,\ y^*(x)\big) \qquad\qquad x \in X$$

$$\text{where} \quad y^*(x) \in \arg\min_{y} \{f(x, y) : y \in Y(x)\}$$

- **Leader** chooses x, anticipating the follower's optimal response
- **Follower** observes x and minimises their own objective
- **Optimistic:** if follower is indifferent, choose the outcome best for the leader

Bilevel Optimisation: Hardness

Computational difficulty

Bilevel optimisation is NP-hard in general [Frangioni 1995].

Our specific challenge

- **Leader** solves an unbounded knapsack problem (NP-hard)
- **Follower** solves parallel machine scheduling $P||C_{max}$ (NP-hard)
- Value function $C^*_{max}(x)$ is non-convex and discontinuous
- No closed-form expression for $C^*_{max}(x)$: every evaluation requires solving the follower, which implies the use of combinatorial approaches (e.g. branch-and-bound) at the upper level

Consequence

General-purpose bilevel solvers (e.g. MibS) can handle only very small instances. We need a **problem-specific exact algorithm**.

The Knapsack-Scheduling Problem

Problem data:

- n job types, m identical parallel machines, budget $B > 0$
- Job type i: processing time $d_i > 0$, cost $p_i > 0$; w.l.o.g. $d_1 \geqslant \cdots \geqslant d_n$
- **Leader** selects $x_i \in \mathbb{Z}_+$ copies of type i; **follower** assigns $y_{ik} \in \mathbb{Z}_+$ jobs of type i to machine k

Complete Bilevel Formulation

$$\begin{aligned} \max_{x \in \mathbb{Z}_+^n} \quad & C^*_{\max}(x) \\ \text{s.t.} \quad & \textstyle\sum_{i=1}^n p_i x_i \leqslant B \end{aligned}$$

$$\text{where} \quad C^*_{\max}(x) = \min_{y_{ik} \in \mathbb{Z}_+} \max_{k=1,\dots,m} \sum_{i=1}^n d_i y_{ik}$$

$$\text{s.t.} \quad \textstyle\sum_{k=1}^m y_{ik} = x_i \quad \forall\, i \in \{1, \dots, n\}$$

Leader **maximises** $C^*_{\max}(x)$; follower **minimises** it — an adversarial bilevel structure.

Why Enumeration Fails

Naive approach: enumerate every feasible x, solve the follower for each.

$$\text{Search space} \leqslant \prod_{i=1}^{n} \left(\left\lfloor \frac{B}{p_i} \right\rfloor + 1 \right)$$

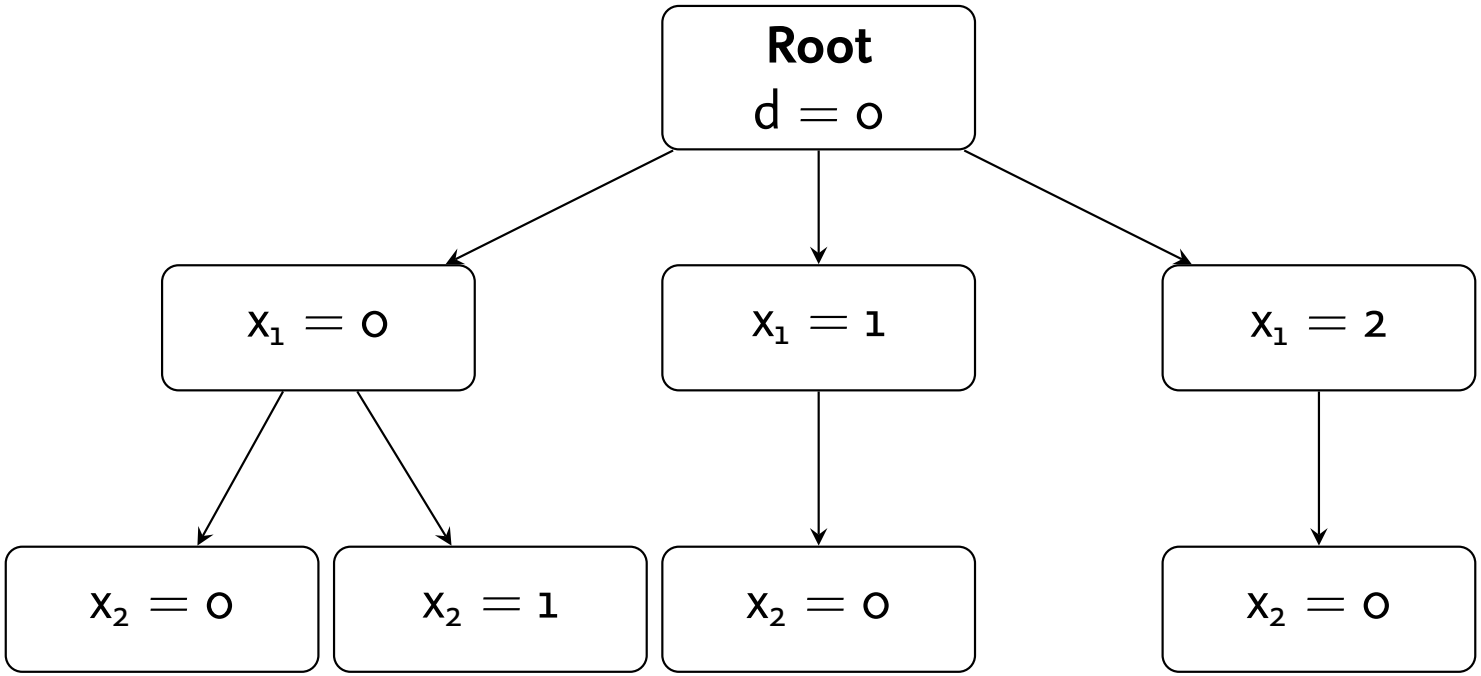
Example: $n = 10$, $B = 100$, $p_i = 5+i$

Upper bound: $\mathbf{5.29 \times 10^{10}}$ candidates

At 1 ms per follower solve: $\approx$ **1.68** years

Solution: Branch-and-Bound with tight upper bounds to prune the tree aggressively.

Search Tree Structure



Example: $n = 2$ types, $p_1 = 5$, $p_2 = 8$, $B = 10$

Each **leaf** requires one follower (scheduling) solve. With large n and B the tree has billions of leaves $\Rightarrow$ intractable.

Branch-and-Bound: Overview

Node structure at depth d

- **Fixed decisions:** $x_1, \ldots, x_d \quad B_{\text{rem}} = B - \sum_{i=1}^{d} p_i x_i$
- **Machine loads:** incremental LPT schedule for items $1, \ldots, d$
- **Children:** $x_{d+1} = 0, 1, \ldots, \lfloor B_{\text{rem}} / p_{d+1} \rfloor$

Three components:

1. **Branching** — fix x_d
2. **Bounding** — compute $\mathrm{UB}(d, B_{\text{rem}})$
3. **Pruning** — discard if $\mathrm{UB} \leqslant C_{\max}^{\text{inc}}$

Design choices:

- Depth-first $\rightarrow$ fast incumbent updates $\rightarrow$ aggressive pruning
- Incremental LPT: $O(m)$ per node
- DP precomputed: $O(1)$ bound lookups

The LPT Rule — Follower's Heuristic

Longest Processing Time (LPT)
[Graham 1969]:

- Sort jobs by non-increasing duration
- Assign each to the least-loaded machine

Guarantee:
$C^*_{\max}(x) \leqslant C^{\mathrm{LPT}}_{\max}(x) \leqslant \frac{4}{3} C^*_{\max}(x)$

Incremental updates

Branching $d \to d{+}1$: append x_{d+1} jobs of duration d_{d+1}. Reuse the existing sorted machine-load state — **no full recompute!**

LPT algorithm

$L_1 = \cdots = L_m \leftarrow 0$
for each job j (non-increasing order):
  $k^* \leftarrow \arg\min_k L_k$
  $L_{k^*} \leftarrow L_{k^*} + d_j$
return $C^{\mathrm{LPT}}_{\max} = \max_k L_k$

Time: $O(J \log m)$

Practice: LPT is optimal in $\approx 84\%$ of our test instances.

Pruning via Upper Bounds

At depth d with remaining budget B_{rem}, we compute an upper bound:

$$C^*_{\max}(x_1, \ldots, x_n) \leqslant \mathrm{UB}(x_1, \ldots, x_d, B_{\text{rem}}) \quad \forall \text{ completions } x_{d+1}, \ldots, x_n$$

Prune whenever $\mathrm{UB} \leqslant C^{\text{inc}}_{\max}$.

$$\mathrm{UB} = \underbrace{C^{\mathrm{LPT},(d)}_{\max}}_{\text{fixed items } 1..d} + \max_{\sum_{i>d} p_i y_i \leqslant B_{\text{rem}}} \underbrace{\Delta(y_{d+1}, \ldots, y_n)}_{\text{extra makespan from remaining items}}$$

Ceiling bound

$\Delta = \sum_{i>d} d_i \lceil y_i / m \rceil$

Single DP table, $O(1)$ lookup

Max-LPT bound

$\Delta = C^{\mathrm{LPT}}_{\max}(y_{d+1}, \ldots, y_n)$

DP table per depth, tighter bound

Ceiling Bound: Formulation

Knapsack Scheduling Relaxation

$$\mathrm{UB} = C_{\max}^{\mathrm{LPT},(d)} + \max_{y \geqslant 0} \sum_{i=d+1}^{n} d_i \left\lceil \frac{y_i}{m} \right\rceil \quad \text{s.t.} \sum_{i>d} p_i y_i \leqslant B_{\mathrm{rem}}$$

Intuition: $\lceil y_i/m \rceil$ jobs of type i on m equal machines raise makespan by $\lceil y_i/m \rceil \cdot d_i$ — always an overestimate.

DP implementation — O(1) lookup per node

Reduce to unbounded knapsack. For item type i:

- **Unit:** cost p_i, value d_i (for $1, \ldots, m-1$ copies)
- **Block:** cost $m \cdot p_i$, value d_i (for a batch of m)

Precompute in reverse item order $\Rightarrow$ any depth-d query is a direct table read.

Ceiling Bound: Validity Proof

Theorem

For any feasible completion: $C^*_{max}(x_1, \ldots, x_n) \leqslant \mathrm{UB}$

Proof sketch:

1. $C^*_{max}(x) \leqslant C^{\mathrm{LPT}}_{max}(x)$ (LPT over-schedules)
2. After fixed items, current makespan $= C^{\mathrm{LPT},(d)}_{max}$
3. Assume all machines filled to $C^{\mathrm{LPT},(d)}_{max}$ — only inflates
4. On m equal-load machines, adding y_i jobs of type i raises makespan by $\lceil y_i/m \rceil \cdot d_i$
5. Therefore $C^{\mathrm{LPT}}_{max}(x) \leqslant C^{\mathrm{LPT},(d)}_{max} + \sum_{i>d} \lceil x_i/m \rceil \cdot d_i$
6. Maximising RHS over B_{rem} gives $\mathrm{UB} \geqslant C^*_{max}(x)$ $\square$

The Max-LPT Problem

Definition

Find $x \in \mathbb{Z}_+^n$ with $\sum_i p_i x_i \leqslant B$ that **maximises** $C_{\max}^{\mathrm{LPT}}(x)$.

Two roles:

1. Solved once $\Rightarrow$ **strong initial incumbent** (with $\frac{3}{4}$-approx. guarantee)
2. Solved for remaining items at each node $\Rightarrow$ **tighter bound** than ceiling

Respects actual LPT structure (not just $\lceil \cdot / m \rceil$).

Key result

Max-LPT solved **optimally** in $O(n \cdot B)$ via DP.

Trade-off

n depth-specific DP tables vs. one ceiling table; recovered by better pruning.

Approximation Guarantee: Building Blocks

Lemma (Properties of α-approximation)

Let $\mathtt{approx}$ be an α-approx. for minimization ($\alpha \geqslant 1$), set $\beta := 1/\alpha$. Then for all x:

$$\beta \cdot \mathrm{Opt}(x) \leqslant \beta \cdot \mathrm{approx}(x) \leqslant \mathrm{Opt}(x) \leqslant \mathrm{approx}(x)$$

Theorem (Bilevel Approx. [Weninger & Fukasawa 2025])

If $\beta \cdot \mathrm{Opt}(x) \leqslant \mathrm{approx}(x) \leqslant \mathrm{Opt}(x) \leqslant \mathrm{approx}(x)$ for all x and $\hat{x} \in \arg\max_x \mathrm{approx}(x)$, then:

$$\beta \cdot \mathrm{Opt}_{\text{bilevel}} \leqslant \mathrm{Opt}(\hat{x}) \leqslant \mathrm{Opt}_{\text{bilevel}}$$

Our setting: LPT is a $\frac{4}{3}$-approx. for $P||C_{max} \Rightarrow \beta = \frac{3}{4}$. Max-LPT maximises $C_{max}^{LPT} \Rightarrow$ apply Theorem directly.

The $\frac{3}{4}$-Approximation Guarantee

Theorem

Let $\hat{x}$ be the Max-LPT solution and x^* the bilevel-optimal solution. Then:

$$\frac{3}{4}\, C^*_{\max}(x^*) \;\leqslant\; C^*_{\max}(\hat{x}) \;\leqslant\; C^*_{\max}(x^*)$$

Proof (using the two building blocks):

1. **Lemma** ($\alpha = \frac{4}{3}$, $\beta = \frac{3}{4}$): for all x, $\frac{3}{4}\, C^*_{\max}(x) \leqslant \frac{3}{4}\, C^{\mathrm{LPT}}_{\max}(x) \leqslant C^*_{\max}(x) \leqslant C^{\mathrm{LPT}}_{\max}(x)$
2. $\hat{x}$ maximises $C^{\mathrm{LPT}}_{\max}$, so $C^{\mathrm{LPT}}_{\max}(x^*) \leqslant C^{\mathrm{LPT}}_{\max}(\hat{x})$
3. **Theorem [WF25]:** $\frac{3}{4} \cdot \mathrm{Opt}_{\mathrm{bilevel}} \leqslant C^*_{\max}(\hat{x}) \leqslant \mathrm{Opt}_{\mathrm{bilevel}}$
4. **Full chain:** $\frac{3}{4}\, C^*_{\max}(x^*) \leqslant \frac{3}{4}\, C^{\mathrm{LPT}}_{\max}(x^*) \leqslant \frac{3}{4}\, C^{\mathrm{LPT}}_{\max}(\hat{x}) \leqslant C^*_{\max}(\hat{x}) \leqslant C^*_{\max}(x^*)$ $\square$

Practical implication

The Max-LPT incumbent is **guaranteed** $\leqslant 25\%$ **suboptimal**. In practice the gap is virtually always zero (LPT is often optimal).

Max-LPT Algorithm: Key Observation

Structural observation

In any LPT schedule, the **last scheduled job** l lands on the **least-loaded machine**:

$$C_{max}^{LPT} = C_{max}^{(m)} + d_l$$

Makespan is determined by the minimum machine load just before l plus its duration d_l.

Two cases for the optimal last job l:

Case 1 — l raises the makespan

Job l lands on $C_{max}^{(m)}$ and becomes the new maximum.

By optimality: no cheaper selection achieves a larger $C_{max}^{(m)}$ under budget $B - p_l$.

$\Rightarrow$ preceding schedule **maximises** $C_{max}^{(m)}$ for items $\leqslant l$ with budget $B - p_l$.

Case 2 — l does not raise the makespan

Some earlier job j set the maximum; all jobs after j leave it unchanged.

The sub-schedule before j must maximise $C_{max}^{(m)}$ at the time j was placed.

$\Rightarrow$ Identical structure — treat j as the last job (Case 1).

Consequence: try every item type J as candidate last job and optimise the rest exactly.

Max-LPT Algorithm: Per-Machine Budget & Properties

Structural observation

In any LPT schedule, the **job that finishes last** I was put on the **least-loaded machine at the time of scheduling**:

$$C_{\max}^{\mathrm{LPT}} = C_{\max}^{(m)} + d_I$$

Makespan is determined by the minimum machine load just before I plus its duration d_I.

For a fixed candidate last job J, we want to maximise $C_{\max}^{(m)}$ for items $1, \ldots, J$.

Two key properties of the optimal preceding schedule

- **Property 1** (highest per-machine budget):
  $B^* := \lfloor B/m \rfloor$ is the largest budget assignable equally to each machine.
  No redistribution can give the least-loaded machine more than B^*.
- **Property 2** (best items for B^*):
  Given budget B^*, the optimal item combination from types $\leqslant J$ is the unbounded knapsack optimum.

If both hold, no schedule achieves a larger $C_{\max}^{(m)}$: buy m copies of every DP-optimal item and run LPT.

Budget adjustment when $\hat{B} := B - mB^* < p_J$

Remainder $\hat{B}$ is too small to pay for one copy of J. Reduce B^* by 1 (uniformly across machines) until $\hat{B} \geqslant p_J$. Property 1 holds for the reduced B^*; re-query $\mathrm{DP}(J, B^*)$. If B^* reaches 0 before $\hat{B} \geqslant p_J$, skip J (infeasible).

Max-LPT Algorithm: Correctness & Complexity

Algorithm (pseudocode)

Precompute DP table (unbounded knapsack, items $1 \dots n$, budgets $0 \dots B$).

$z^* \leftarrow 0$ // best candidate so far

for each type $J \in \{1, \dots, n\}$:

  $B^* \leftarrow \lfloor B/m \rfloor$

  while $(B - mB^*) < p_J$ **and** $B^* > 0$: $B^* \leftarrow B^* - 1$

  if $(B - mB^*) < p_J$: skip J // infeasible

  $z_J \leftarrow \mathrm{DP}(J, B^*) + d_J$; $z^* \leftarrow \max(z^*, z_J)$

return z^*

Correctness

Any optimal solution has a last job I matching Case 1 or 2.

For that I, Properties 1 & 2 identify the optimal preceding schedule via $\mathrm{DP}(I, B^*)$.

Taking $\max_J z_J$ over all candidates covers it. □

Complexity: $O(n \cdot B)$

DP table precomputed in $O(n \cdot B)$.

Per candidate J: $O(1)$ lookup + $O(B^*/m)$ adjustment.

n candidates $\Rightarrow O(n \cdot B)$ total.

Early Optimality Detection

Key inequality

For all x: $C^*_{max}(x) \leqslant C^{LPT}_{max}(x)$ (LPT only overestimates)

Let $\hat{x} \in \arg\max_x C^{LPT}_{max}(x)$. If $C^{LPT}_{max}(\hat{x}) = C^*_{max}(\hat{x})$, then for any feasible x:

$$C^*_{max}(\hat{x}) = C^{LPT}_{max}(\hat{x}) \geqslant C^{LPT}_{max}(x) \geqslant C^*_{max}(x)$$

$\Rightarrow$ $\hat{x}$ is the **global bilevel optimum** - no B&B tree needed!

Impact

- $C^*_{max}(\hat{x})$ is computed anyway to initialise the incumbent
- This check costs **nothing extra**
- Triggered in $\approx$ **84.6**% of our test instances $\Rightarrow$ solved at root

Complete Algorithm

1. Solve Max-LPT $\rightarrow \hat{x}$; compute $C_{max}^{LPT}(\hat{x})$, $C_{max}^{*}(\hat{x})$
2. **If** $C_{max}^{LPT}(\hat{x}) = C_{max}^{*}(\hat{x})$: **return** $\hat{x}$
3. $C_{max}^{inc} \leftarrow C_{max}^{*}(\hat{x})$; push root to stack Q
4. **While** $Q \neq \emptyset$:
   - Remove node $(d, x_1, \ldots, x_d, B_{rem})$
   - If $d = n$: solve follower; update C_{max}^{inc}
   - Else: compute $UB(d, B_{rem})$
   - If $UB \leqslant C_{max}^{inc}$: **prune**
   - Else: push children (depth-first)
5. Return incumbent

Inspired by [Weninger & Fukasawa 2025].

Leaf optimisation

At depth $n-1$: only the child spending maximum remaining budget matters. All others produce a smaller or equal C_{max}^{*}.

Follower solver

Gurobi 12.0.3 [Gurobi Optimization 2024] solves the MIP formulation of $P||C_{max}$ at leaves.

Experimental Setup

Hardware & software:

- AMD Ryzen 5 9600X, 32 GB RAM
- Python 3.11, Gurobi 12.0.3
- Single-threaded runs

PatternTestSet: 140 instances

7 structural patterns:
Uniform Ratios, High Variance, Increasing, Random Moderate, Extreme, Strong Correlation, Subset Sum

6–12 types, 3–8 machines, budget 15–50

Correctness: vs. enumeration

- 140 instances: **0 discrepancies**
- Enumeration timed out on 15/140
- B&B solved all 140 in ms

Speed-up

Up to $\mathbf{10^6} \times$ faster than complete enumeration

Grid Sensitivity Analysis

Grid: 6×6 (types × machines), 3 budget multipliers, 5 seeds = 540 instances; 1 h limit.

Types: 4, 8, 12, 16, 20, 24 • **Machines:** 2, 4, 6, 8, 10, 12 • **Multipliers:** 1.3, 2.5, 5.0

Budget ×	Root Opt.	Completed	Timeout	Leaf T/O
1.3×	154 / 154	22 / 26	2 / 0	2 / 0
2.5×	150 / 150	19 / 22	3 / 0	8 / 8
5.0×	153 / 153	20 / 23	4 / 1	3 / 3

Ceil / MaxLPT

Main finding

84.6% of instances solved at root — no B&B tree needed

MaxLPT advantage

More completions and fewer timeouts in every setting

Ceiling vs. MaxLPT: Hard Instances

61 non-root-optimal instances where both bounds completed within 1 h:

Runtime (Ceiling − MaxLPT):

- Mean: +71 s (ceiling slower)
- Median: +0.4 s
- Max: +1,057 s
- MaxLPT faster: **41 of 61 cases**

Nodes (Ceiling − MaxLPT):

- Mean: +4.8M nodes
- Median: +69K
- Max: +54.5M
- MaxLPT always fewer nodes

Interpretation

MaxLPT prunes more aggressively on hard instances. Precomputing n DP tables is fully justified by the reduced node count. Ceiling is marginally faster for trivial (root-optimal) cases.

Comparison with MibS

MibS: general-purpose bilevel MILP solver [Tahernejad et al. 2020]
Subset: $m \in \{4, 8, 12\}$, $j \in \{4, 8, 16, 20\}$, 3 multipliers, 5 seeds $=$ 180 instances

$m \setminus j$	4	8	16	20
4	15/15	8/15	2/15	1/15
8	5/15	0/15	0/15	1/15
12	2/15	0/15	0/15	0/15

MibS completions (of 15) per cell. MaxLPT B&B: **179/180**.

Speed-up over MibS (1 h limit)

Ceiling: mean +2,876.6 s, median +3,599.9 s MaxLPT: mean +2,959.6 s, median +3,599.5 s
Problems solved by our algorithm in milliseconds require **over an hour** for MibS.

Summary & Conclusions

Contributions:

1. Formal bilevel knapsack-scheduling model
2. Ceiling bound with precomputed $O(1)$ DP, solvable in $O(n \cdot B)$
3. Max-LPT solved optimally in $O(n \cdot B)$
4. $\frac{3}{4}$-approximation guarantee (theory)
5. Early optimality detection (84.6% solve at root)
6. Comprehensive computational study

Outperforms MibS by **orders of magnitude**.

Limitations

- No characterisation of easy instances
- Identical machines only

Future work

- Related/unrelated machines
- Stochastic processing times
- Characterise root-optimal instances

Thank you!

Questions?

Ole Damke • o.damke@tu-braunschweig.de
Institut für Mathematische Optimierung • TU Braunschweig

References I

- A. Frangioni. On a new class of bilevel programming problems and its use for reformulating mixed integer problems. Eur. J. Oper. Res., 82(3):615–646, 1995.
- R. L. Graham. Bounds on Multiprocessor Timing Anomalies. SIAM J. Appl. Math., 17(2):416–429, 1969.
- N. Weninger, R. Fukasawa. A fast combinatorial algorithm for the bilevel knapsack problem. Math. Program., 210(1):847–879, 2025.
- S. Tahernejad, T. K. Ralphs, S. T. DeNegre. A branch-and-cut algorithm for mixed integer bilevel linear optimisation. Math. Program. Comput., 12(4):529–568, 2020.
- T. Kleinert et al. A survey on MIP techniques in bilevel optimisation. EURO J. Comput. Optim., 9:100007, 2021.
- S. Dempe, A. Zemkoho. Bilevel Optimization, vol. 161. Springer, 2020.

References II

Gurobi Optimization, LLC. Gurobi Optimizer Reference Manual. Version 12.0, 2024. https://www.gurobi.com